Colégio Técnico de Campinas - COTUCA
25/04/2024

Projeto De Galeria Virtual
Relatório De Desenvolvimento Do Projeto De Galeria Virtual

Disciplina: 19 – Informática Diurno
1º semestre de 2024
Prof. Dr. Francisco da Fonseca Rodrigues

Alunos:
Miguel Oliveira de Queiroz - 24143
Henrique Hideki Inoue - 24528

Iniciamos o projeto desenvolvendo um esqueleto geral do projeto, começando pelas classes durante o primeiro dia, havendo erros de declaração nos atributos. Depois seguimos para o programa principal e a estruturação do seletor principal.

```python
import tkinter, classes, os

def main():
    selected = "Start"
    while selected != 0:
        options()
        selected = int(input(""))
        match selected:
            case 1:
                func1()
            case 2:
                func2()
            case 3:
                func3()
            case 4:
                func4()
        os.system("cls") or None

def options():
    print("1. Cadastro de obras de arte")
    print("2. Listagem de obras de arte")
    print("3. Página web de obras de arte")
    print("4. Triangulo de Pascal")
    print("0. Terminar a execução do programa")

def func1():
    input("Pressione [ENTER] Para Continuar!")
```

```python
class Obra():
    def __init__(self):
        self.ano_obra[4] = ""
        self.mes_obra[2] = ""
        self.autor_obra[20] = ""
        self.nome_obra[20] = ""
        self.estilo_obra[15] = ""
        self.url_obra[100] = ""
        self.valor_estimado : float = 0
        self._aberto_para_gravação : bool = 0
        self._arquivo = open(r"")

    def lerCamposDoArquivo(self):
        pass

    def gravarCamposNoArquivo(self):
        pass

    def preencherCampos(self):
        pass

    def fecharArquivo(self):
        pass

    def __str__(self):
        pass
```

Após terminar o seletor, nós reestruturamos os atributos da classe, adicionamos o método compararCom(+parâmetros) e adicionamos parâmetros aos métodos preencherCampos() e __init__(). Identificamos dificuldades na leitura/escrita do arquivo, já que a abertura dele sob este contexto implica no não funcionamento de fecharArquivo()

```python
class Obra():
    def __init__(self, nomeArq : str, paraEscrever : bool):
        self.ano_obra : str[4] = ""
        self.mes_obra : str[2] = ""
        self.autor_obra : str[20] = ""
        self.nome_obra : str[20] = ""
        self.estilo_obra : str[15] = ""
        self.url_obra : str[100] = ""
        self.valor_estimado : float = 0
        self._aberto_para_gravação : bool = paraEscrever
        self._arquivo = nomeArq

    def lerCamposDoArquivo(self):
        pass

    def gravarCamposNoArquivo(self):
        if self._aberto_para_gravação:
            file = open(self._arquivo, "a")
            file.write(self.__str__())
            file.close()

    def preencherCampos(self, novoAno, novoMes, novoAutor, novoNome, novoEstilo, novoValor, novaURL : str):
        pass

    def fecharArquivo(self):
        pass

    def __str__(self):
        string = ""
        return string
```

O método __str__() foi relativamente chato de padronizar, já que a formatação é muito mecânica e erros relacionados às f-strings são relativamente chatos de corrigir seguindo o padrão descrito.

É claro, não perdemos um segundo se quer e começamos o desenvolvimento das duas primeiras funções em main(). Identificamos erros múltiplas vezes em relação ao uso das tuplas no tkinter (“_tkinter.TclError: bad file type "*.txt"...”), o que foi arrumado usando [(valores_tupla)] ao invés de (valores_tupla) e em “if ask”, que mudamos para “if len(ask) == 0”.

```python
def func1():
    ask = filedialog.askopenfilename(title="James", initialdir="C:/Documents/", multiple=False, filetypes=[("Arqu
    if ask == "":
        print("Erro Na Abertura De Arquivo!")
        return 1
    call = classes.Obra(ask, 1)
    call.preencherCampos()
    call.gravarCamposNoArquivo()
    input("Pressione [ENTER] Para Continuar!")

def func2():
    ask = filedialog.askopenfilename(title="James", initialdir="C:/Documents/", multiple=False, filetypes=[("Arqu
    if ask == "":
        print("Erro Na Abertura De Arquivo!")
        return 1
    call = classes.Obra(ask, 0)
    call.lerCamposDoArquivo()
    input("Pressione [ENTER] Para Continuar!")
```

Além disso, inserimos laços de repetição na func1() e func2(), para permitir a leitura completa da lista e cadastro seguido de vários valores. Inserimos a estrutura geral de uma página html na func3() para podermos adicionar os laços repetitivos maiores futuramente.

```
def func3():
    ask = filedialog.askopenfilename(title="Selecione Um Arquivo De Obras", multiple=False, filetypes=[("Arquivo
    if len(ask) == 0:
        print("Erro Na Abertura De Arquivo!")
        return 1
    call = classes.Obra(ask, 0)
    arquivo = open("obras.html", "w")
    arquivo.write("<!DOCTYPE html>\n")
    arquivo.write("<html lang = ""pt-br"">\n")
    arquivo.write("<head>\n")
    arquivo.write("<title>James</title>\n")
    arquivo.write("</head>\n")
    arquivo.write("<body>\n")
    arquivo.write("<table>\n")
    arquivo.write("<tr><th>Relatório De Obras Da Galeria Virtual</th></tr>\n")
    info = "0"
    while info != 1:
        info = call.lerCamposDoArquivo()
        arquivo.write(f"<tr><td>{call.ano_obra} / {call.mes_obra}</td><td>{call.nome_obra}</td><td>{call.estilo_o
    arquivo.write("</html>\n")
    arquivo.close()
    call.fecharArquivo()
    input("Pressione [ENTER] Para Continuar!")
```

Para desenvolvermos a classe Matemática e a func4(), foi necessário pensar bastante fora da caixa, já que as primeiras soluções se baseavam em usar listas, as quais dificultavam a leitura e o processamento dos dados. Depois de algumas iterações, esse foi o resultado final:

```
    def __init__(self , numero):
        self._numeroBase : int = numero #pensar em um nome melhor

    def fatorial(self, x : int) -> int:
        fatorial_calculado = 1
        while x >= 2:
            fatorial_calculado = x * fatorial_calculado

    def triangulo_de_Pascal(self):
        n = 0
        k = 0
        linha_string = ""
        triangulo = ""
        while n < self._numeroBase:
            while k <= n:
                if k == 0 or k == n:
                    linha_string += "1".ljust(6)
                    k += 1
                else:
                    numero_calculado = self.fatorial(n) // (self.fatorial(k) * self.fatorial(n - k))
                    linha_string += f"{numero_calculado}".ljust(6)
                    k += 1
            n += 1
            k = 0
            triangulo += linha_string + "\n"
            linha_string = ""
        return triangulo
```

Já na reta final do projeto, com a maioria das coisas no lugar e os erros mais grotescos resolvidos, entravam os loops mais compostos, o que torna o processo muito mais preciso, o que associado à lógica necessária para desenvolver a aplicação, tornou essa etapa final muito mais lenta do que o esperado. Os últimos dias do projeto foram direcionados a aperfeiçoamentos dos mecanismos de abertura do navegador, soma do

“valor parcial” de cada ano das obras, adição de css simples à página, e resoluções na formatação da func1() e func2().

O projeto foi feito com intenção de seguir o máximo das orientações, mas houveram adaptações mínimas em algumas instâncias. O projeto foi desenvolvido durante 2 semanas. Estivemos presentes em dois períodos de monitoria para garantir a qualidade do projeto.

Referências:

GOGH, V. **Autorretrato**, 1887. Acesso Em: <https://www.artic.edu/artworks/80607/self-portrait/>
MACCARI, J. **A Musa**, 2016. Acesso Em: <https://www.jercimaccari.com.br/site/loja/8/a-musa/>
MACCARI, J. **Conversa Fiada I**, 2016. Acesso Em: <https://www.jercimaccari.com.br/site/loja/40/conversa-fiada-i/>
MACCARI, J. **Cultura do Figo I**, 2018. Acesso Em: <https://www.jercimaccari.com.br/site/loja/46/cultura-do-figo-i/>
MACCARI, J. **Capoeiristas**, 2023. Acesso Em: <https://www.jercimaccari.com.br/site/loja/197/capoeiristas/>